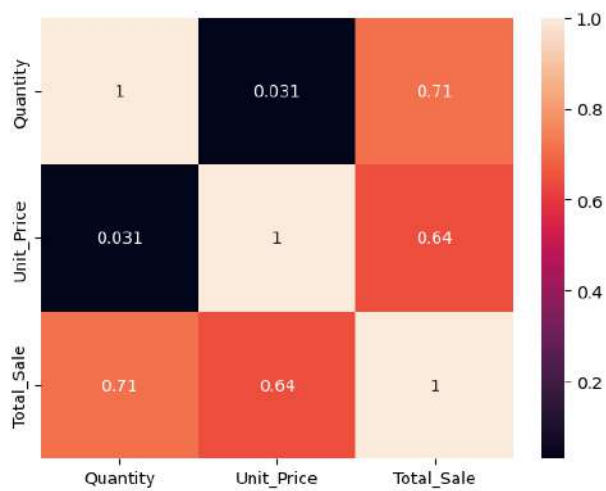


df.describe()

	Row ID	Order Date	Ship Date	Postal Code	Sales	Quantity	Discount	Profit
count	2323.000000	2323	2323	2323.000000	2323.000000	2323.000000	2323.000000	2323.000000
mean	5030.547998	2015-04-29 19:22:17.408523520	2015-05-03 20:45:58.501937152	65340.572536	215.772661	3.779595	0.240353	17.092709
min	15.000000	2013-01-03 00:00:00	2013-01-07 00:00:00	46060.000000	0.444000	1.000000	0.000000	-3701.892800
25%	2564.500000	2014-04-29 00:00:00	2014-05-05 00:00:00	55125.000000	14.620000	2.000000	0.000000	-5.664300
50%	5029.000000	2015-07-23 00:00:00	2015-07-27 00:00:00	65203.000000	45.980000	3.000000	0.200000	5.184000
75%	7612.500000	2016-05-09 12:00:00	2016-05-13 00:00:00	77041.000000	200.012000	5.000000	0.300000	22.456200
max	9984.000000	2016-12-30 00:00:00	2017-01-05 00:00:00	79907.000000	17499.950000	14.000000	0.800000	8399.976000
std	2923.638538	NaN	NaN	11427.949045	632.779010	2.160561	0.265433	291.487465

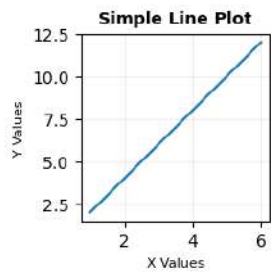
```
num_data=df_comprehensive[["Quantity", "Unit_Price", "Total_Sale"]]  
cor=num_data.corr()  
import seaborn as sns  
sns.heatmap(cor,annot=True)
```

<Axes: >



```
# Simple Line plot
x = np.array([1, 2, 3, 4, 5, 6])
y = np.array([2, 4, 6, 8, 10, 12])

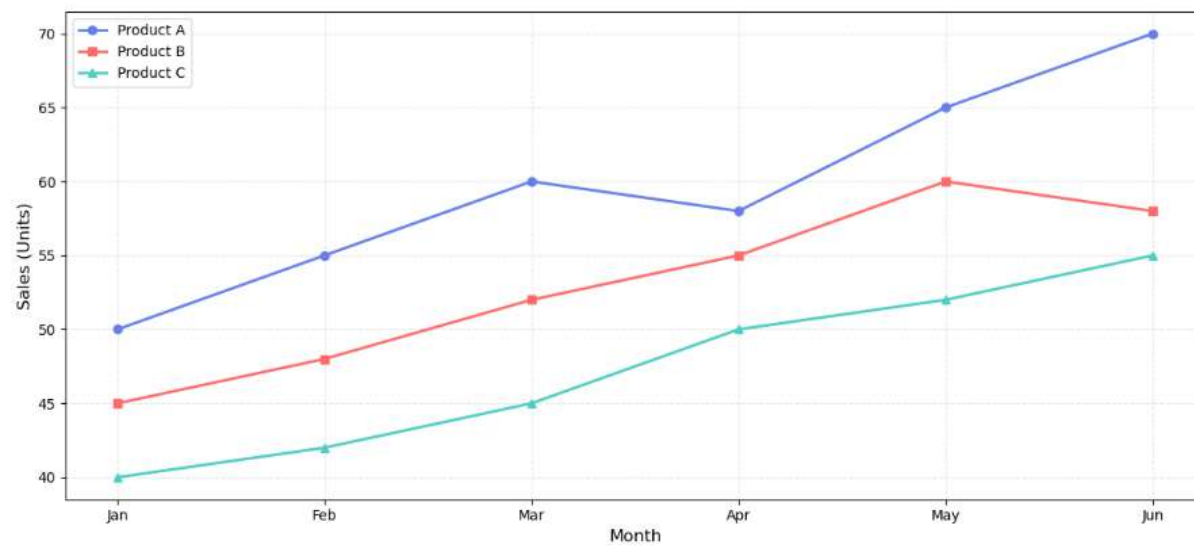
plt.figure(figsize=(2, 2))
plt.plot(x, y)
plt.title('Simple Line Plot', fontsize=10, fontweight='bold')
plt.xlabel('X Values', fontsize=8)
plt.ylabel('Y Values', fontsize=8)
plt.grid(True, alpha=0.2)
plt.show()
```



```
# Multiple lines with styling
months = np.array(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'])
product_a = np.array([50, 55, 60, 58, 65, 70])
product_b = np.array([45, 48, 52, 55, 60, 58])
product_c = np.array([40, 42, 45, 50, 52, 55])
plt.figure(figsize=(12, 6))
plt.plot(months, product_a, marker='o', linewidth=2, label='Product A', color='#667eea')
plt.plot(months, product_b, marker='s', linewidth=2, label='Product B', color='#ff6b6b')
plt.plot(months, product_c, marker='^', linewidth=2, label='Product C', color='#4ecdc4')

plt.title('Monthly Sales Comparison', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Month', fontsize=12)
plt.ylabel('Sales (Units)', fontsize=12)
plt.legend(loc='upper left', fontsize=10)
plt.grid(True, alpha=0.3, linestyle='--')
plt.tight_layout()
plt.show()
```

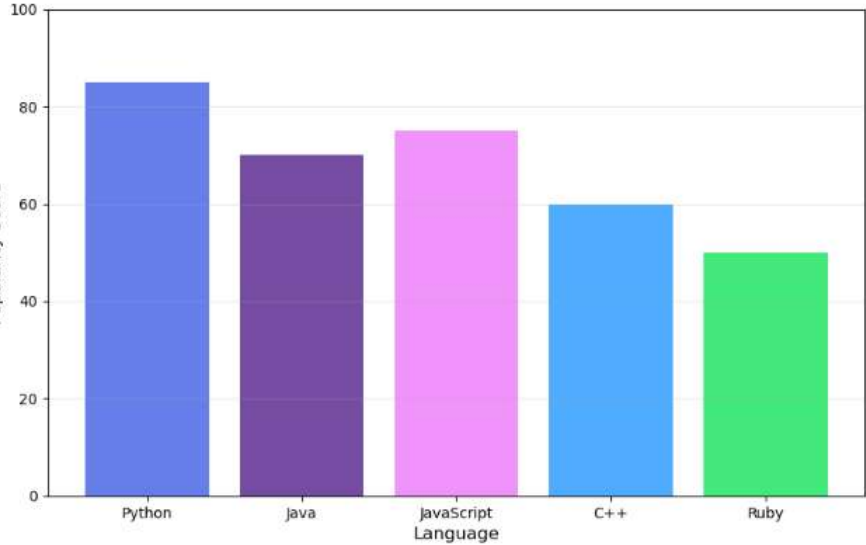
Monthly Sales Comparison



```
# Simple bar chart
categories = ['Python', 'Java', 'JavaScript', 'C++', 'Ruby']
popularity = [85, 70, 75, 60, 50]

plt.figure(figsize=(10, 6))
bars = plt.bar(categories, popularity, color=['#667eea', '#764ba2', '#f093fb', '#4facfe', '#43e97b'])
plt.title('Programming Language Popularity', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Language', fontsize=12)
plt.ylabel('Popularity Score', fontsize=12)
plt.ylim(0, 100)
plt.grid(axis='y', alpha=0.3)
```

Programming Language Popularity

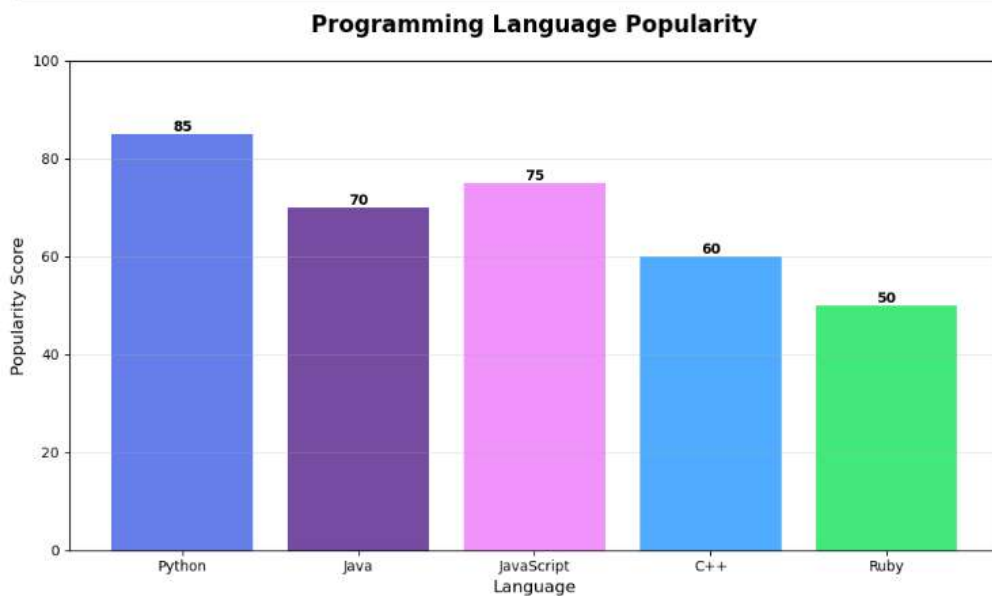


```
# Simple bar chart
categories = ['Python', 'Java', 'JavaScript', 'C++', 'Ruby']
popularity = [85, 70, 75, 60, 50]

plt.figure(figsize=(10, 6))
bars = plt.bar(categories, popularity, color=['#667eea', '#764ba2', '#f093fb', '#4facfe', '#43e97b'])
plt.title('Programming Language Popularity', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Language', fontsize=12)
plt.ylabel('Popularity Score', fontsize=12)
plt.ylim(0, 100)
plt.grid(axis='y', alpha=0.3)

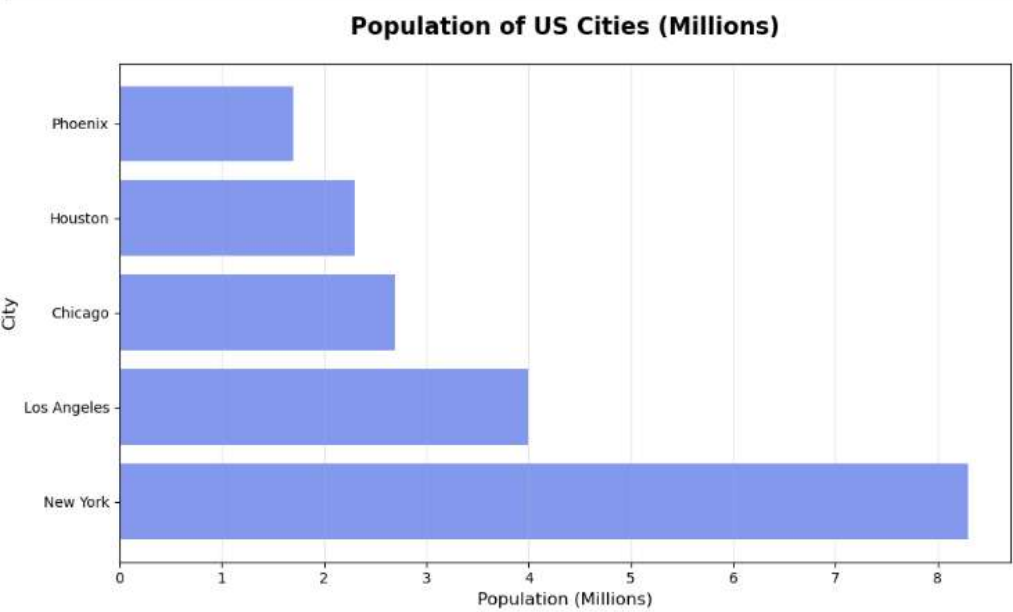
# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2., height,
             f'{height}',
             ha='center', va='bottom', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.show()
```



```
# Horizontal bar chart
cities = ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix']
population = [8.3, 4.0, 2.7, 2.3, 1.7] # in millions

plt.figure(figsize=(10, 6))
plt.barh(cities, population, color='#6677ee', alpha=0.8)
plt.title('Population of US Cities (Millions)', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Population (Millions)', fontsize=12)
plt.ylabel('City', fontsize=12)
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```

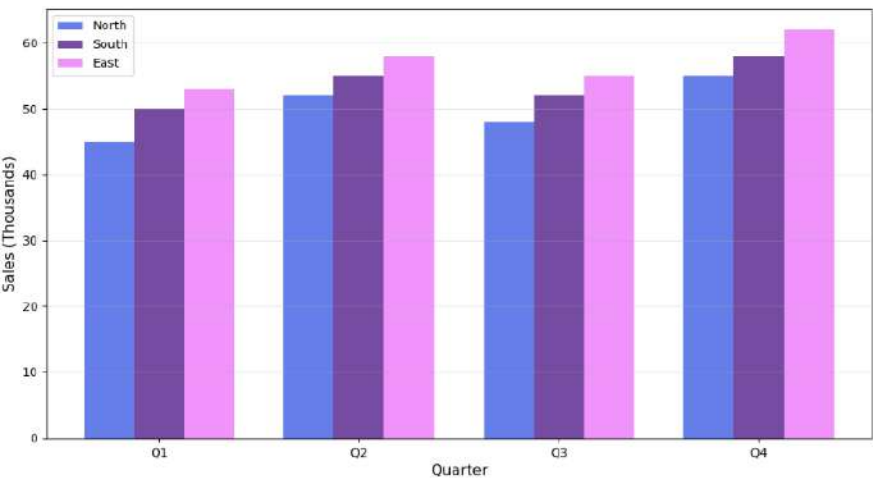


```
# Grouped bar chart
x = np.arange(4) # -> [0, 1, 2, 3]
width = 8.25
q1 = [45, 52, 48, 55]
q2 = [50, 55, 52, 58]
q3 = [53, 58, 55, 62]
quarters = ['Q1', 'Q2', 'Q3', 'Q4']

plt.figure(figsize=(10, 6))
plt.bar(x - width, q1, width, label='North', color='#6677aa')
plt.bar(x, q2, width, label='South', color='#7766aa')
plt.bar(x + width, q3, width, label='East', color='#aa66ff')

plt.title('Quarterly Sales by Region', fontsize=15, fontweight='bold', pad=20)
plt.xlabel('Quarter', fontsize=12)
plt.ylabel('Sales (Thousands)', fontsize=12)
plt.xticks(x, quarters)
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

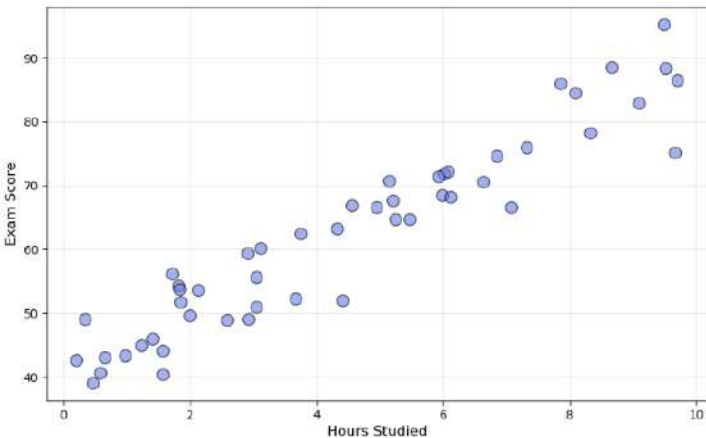
Quarterly Sales by Region




```
# Generate sample data - hours studied vs exam scores
np.random.seed(12)
hours_studied = np.random.uniform(0, 10, 50)
exam_scores = 40 + 5 * hours_studied + np.random.normal(0, 5, 50)

plt.figure(figsize=(10, 6))
plt.scatter(hours_studied, exam_scores, c='#6677aa', alpha=0.6, s=100, edgecolor='black', linewidth=1)
plt.title('Relationship: Study Hours vs Exam Scores', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Hours Studied', fontsize=12)
plt.ylabel('Exam Score', fontsize=12)
plt.grid(True, alpha=0.5)
plt.show()
```

Relationship: Study Hours vs Exam Scores



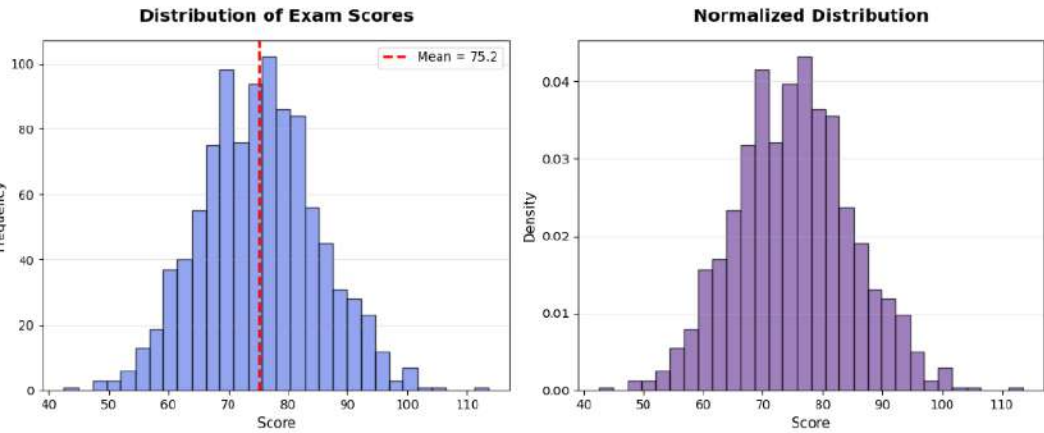
```
# Generate sample data - exam scores
np.random.seed(42)
scores = np.random.normal(75, 10, 1000) # mean=75, std=10

plt.figure(figsize=(12, 5))

# Histogram
plt.subplot(1, 2, 1)
plt.hist(scores, bins=30, color='#6677aa', alpha=0.7, edgecolor='black')
plt.title('Distribution of Exam Scores', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Score', fontsize=11)
plt.ylabel('Frequency', fontsize=11)
plt.axvline(scores.mean(), color='red', linestyle='--', linewidth=2, label=f'Mean = {scores.mean():.1f}')
plt.legend()
plt.grid(axis='y', alpha=0.3)

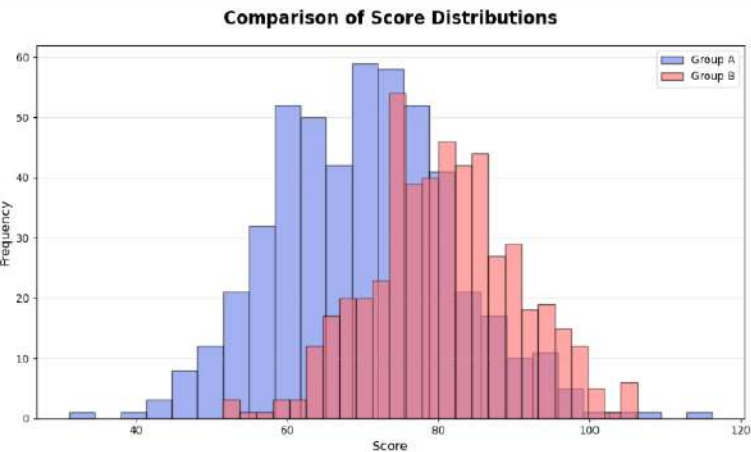
# Histogram with density (normalized)
plt.subplot(1, 2, 2)
plt.hist(scores, bins=30, color='#778899', alpha=0.7, edgecolor='black', density=True)
plt.title('Normalized Distribution', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Score', fontsize=11)
plt.ylabel('Density', fontsize=11)
plt.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



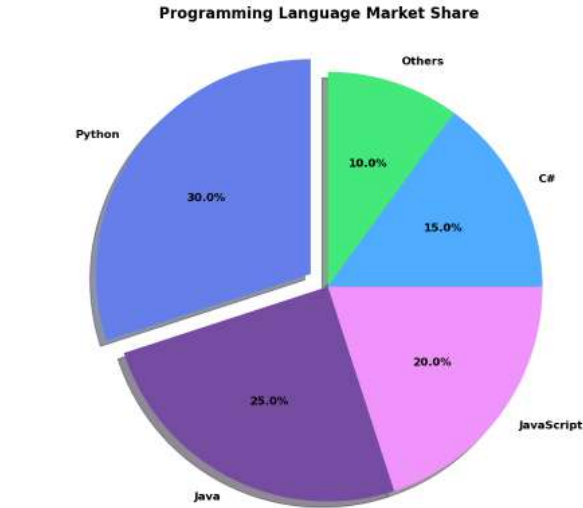
```
# Multiple histograms
group_a = np.random.normal(70, 12, 500)
group_b = np.random.normal(80, 18, 500)

plt.figure(figsize=(10, 6))
plt.hist(group_a, bins=25, alpha=0.8, label='Group A', color='#66b3ff', edgecolor='black')
plt.hist(group_b, bins=25, alpha=0.8, label='Group B', color='#ff9999', edgecolor='black')
plt.title('Comparison of Score Distributions', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Score', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.legend()
plt.grid(axis='y', alpha=0.5)
plt.tight_layout()
plt.show()
```



```
# Market share data
labels = ['Python', 'Java', 'JavaScript', 'C#', 'Others']
sizes = [30, 25, 20, 15, 10]
colors = ['#8B734D', '#708090', '#4682B4', '#FFA500', '#3CB371']
explode = (0.1, 0, 0, 0, 0) # Explode the first slice

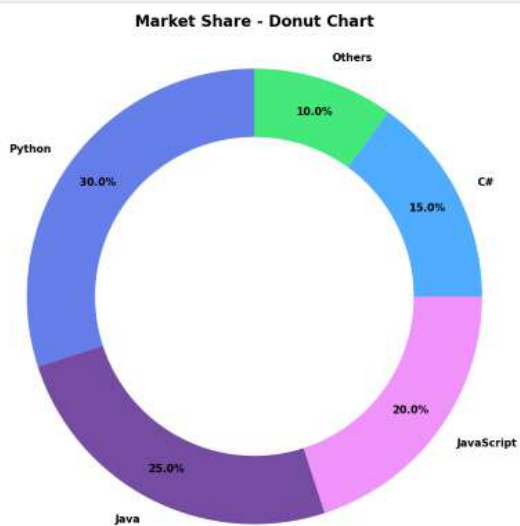
plt.figure(figsize=(10, 8))
plt.pie(sizes, explode=explode, labels=labels, colors=colors, autopct='%1.1P',
        shadow=True, startangle=90, textprops={'fontSize': 12, 'fontWeight': 'bold'})
plt.title('Programming Language Market Share', fontsize=14, fontweight='bold', pad=20)
plt.axis('equal') # Equal aspect ratio ensures the circular pie
plt.tight_layout()
plt.show()
```



```
# Donut chart
fig, ax = plt.subplots(figsize=(10, 8))
ax.pie(sizes, labels=labels, colors=colors, autopct='%2.1f%%',
      startangle=90, pctdistance=0.85, textprops={'fontsize': 11, 'fontweight': 'bold'})

# Draw circle in center to create donut
centre_circle = plt.Circle((0, 0), 8.78, fc='white')
fig.gca().add_artist(centre_circle)

ax.set_title('Market Share - Donut Chart', fontsize=18, fontweight='bold', pad=20)
ax.axis('equal')
plt.tight_layout()
plt.show()
```



```
# Import math
import math
x = 0.1:0.001:10, 1000
y1 = sin(x)
y2 = cos(x)
y3 = sin(x) + cos(x)
y4 = sin(x) * cos(x)

# Create the plot of sin(x)
fig, axes = plt.subplots(2, 2, figsize=(10, 10))
fig.suptitle('Mathematical Functions Visualization', fontsize=16, fontweight='bold', y=1.02)

# Plot 1: sin(x)
axes[0, 0].plot(x, y1, color='blue', linestyle='solid')
axes[0, 0].set_title('sin(x)', fontsize=12, fontweight='bold')
axes[0, 0].set_xlabel('x')
axes[0, 0].set_ylabel('y')
axes[0, 0].grid(True, alpha=0.5)

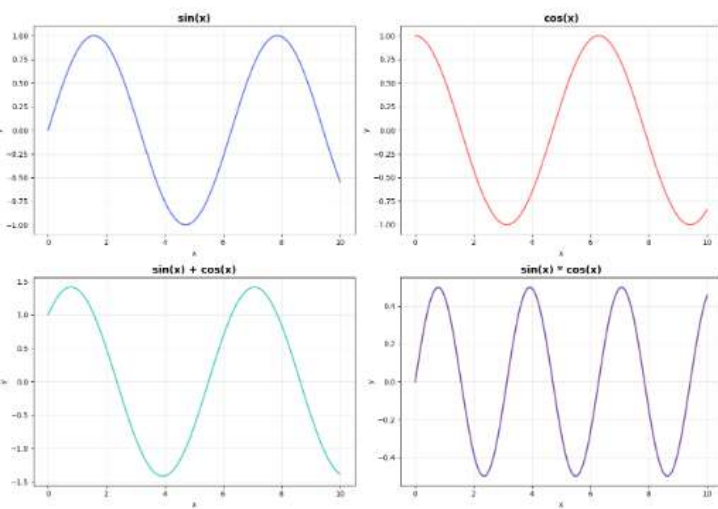
# Plot 2: cos(x)
axes[0, 1].plot(x, y2, color='red', linestyle='solid')
axes[0, 1].set_title('cos(x)', fontsize=12, fontweight='bold')
axes[0, 1].set_xlabel('x')
axes[0, 1].set_ylabel('y')
axes[0, 1].grid(True, alpha=0.5)

# Plot 3: sin(x) + cos(x)
axes[1, 0].plot(x, y3, color='cyan', linestyle='solid')
axes[1, 0].set_title('sin(x) + cos(x)', fontsize=12, fontweight='bold')
axes[1, 0].set_xlabel('x')
axes[1, 0].set_ylabel('y')
axes[1, 0].grid(True, alpha=0.5)

# Plot 4: sin(x) * cos(x)
axes[1, 1].plot(x, y4, color='purple', linestyle='solid')
axes[1, 1].set_title('sin(x) * cos(x)', fontsize=12, fontweight='bold')
axes[1, 1].set_xlabel('x')
axes[1, 1].set_ylabel('y')
axes[1, 1].grid(True, alpha=0.5)

plt.tight_layout()
plt.show()
```

Mathematical Functions Visualization



```
# generate comprehensive sales data
np.random.seed(42)

# Data
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
revenue = [1000, 1200, 1500, 1800, 2000, 2200]
expenses = [500, 550, 600, 650, 700, 750]
profit = [500, 650, 900, 1150, 1300, 1450]

products = ['Electronics', 'Clothing', 'Food', 'Books', 'Others']
product_sales = [10, 20, 30, 15, 5]

customer_age = np.random.normal(35, 12, 100)

# Create dashboard
fig = plt.figure(figsize=(16, 10))
fig.suptitle('Sales Analytics Dashboard - Q1 & Q2 2024', fontsize=16, fontweight='bold', y=0.98)

# 1. Revenue vs Expenses (Line Chart)
ax1 = plt.subplot(2, 1, 1)
ax1.plot(months, revenue, marker='x', linestyle='-', label='Revenue', color='blue')
ax1.plot(months, expenses, marker='x', linestyle='-', label='Expenses', color='red')
ax1.set_title('Revenue vs Expenses', fontweight='bold', fontsize=14, y=0.95)
ax1.set_xlabel('Month')
ax1.set_ylabel('Amount ($)')
ax1.legend()
ax1.grid(True, alpha=0.3)
```

Sales Analytics Dashboard - Q1 & Q2 2024

